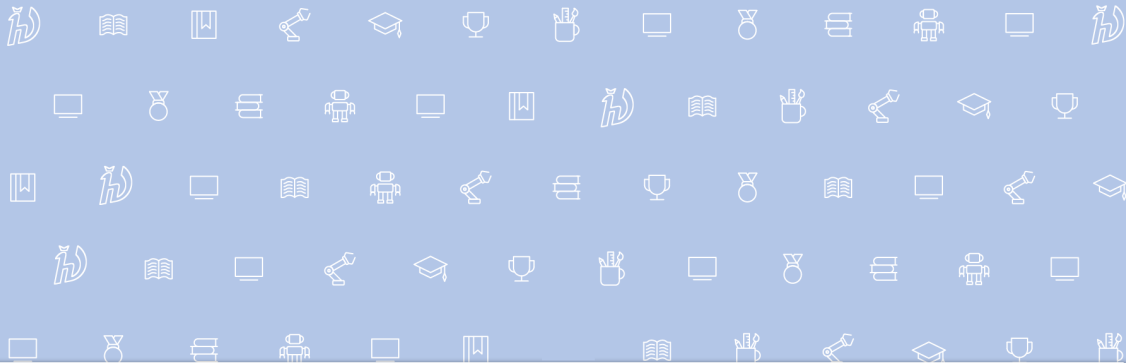




Artificial Intelligence Software, Lecture 7

Training process



Center for
Cognitive
Modeling



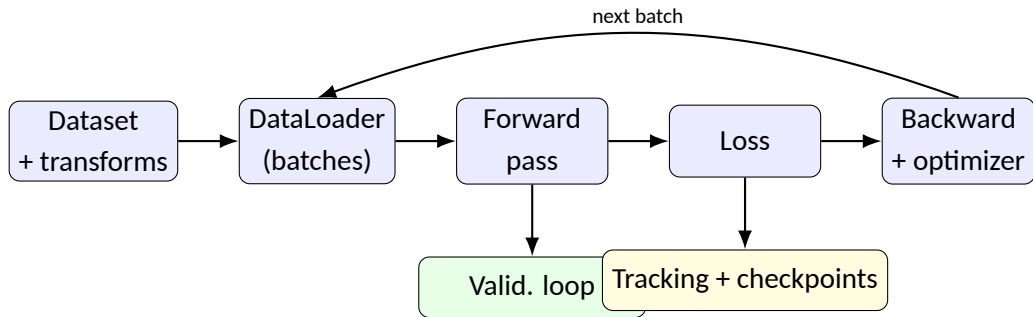
Training as a Pipeline

Training a neural network is a production pipeline, not one monolithic algorithm.

- Input stage: data loading, filtering, augmentation, batching.
- Compute stage: forward pass, loss computation, backward pass, optimizer step.
- Control stage: validation, checkpointing, logging, early stopping.
- Ops stage: experiment tracking, distributed execution, model registry.

Your work is to design and control this system, not to re-implement autograd.

Pipeline Scheme





Key Idea: Build on Existing Models

In practical ML, we rarely train foundation models from scratch.

- Start with pretrained encoders/LLMs/vision backbones.
- Adapt with fine-tuning, adapters, or LoRA-style parameter-efficient methods.
- Keep a consistent pipeline: same data contracts, metrics, and deployment checks.

Engineering maturity comes from reproducibility and iteration speed.



Fine-Tuning Strategies

- **Full fine-tuning:** best quality ceiling, highest memory/compute cost.
- **Partial unfreezing:** train only top layers for faster adaptation.
- **PEFT (LoRA/adapters):** small trainable modules, cheap and scalable.

Typical workflow:

1. Freeze most layers.
2. Train head or adapters.
3. Unfreeze gradually if validation metric plateaus.



Data and Dataset Quality

Dataset quality determines your quality ceiling.

- Remove duplicates and label leakage between train/validation/test.
- Control class imbalance and long-tail behavior.
- Keep metadata for slicing metrics (domain, source, language, device).
- Version data exactly as you version code.

Bad labels are often a bigger bottleneck than model architecture.



DataLoader Design Decisions

DataLoader converts storage format into GPU-ready batches.

- Throughput: prefetch, pinned memory, parallel workers.
- Stochasticity: shuffling and sampling strategy.
- Consistency: deterministic seeds for reproducible debugging.
- Efficiency: dynamic padding or bucketing for variable-length inputs.



Batching and Effective Batch Size

- Physical batch size is limited by GPU memory.
- Effective batch size can be increased via gradient accumulation.
- Very large batches may require learning-rate scaling and warmup.

effective batch = batch per device $\times$ num devices $\times$ accumulation steps



Forward Pass and Loss

- Forward pass transforms batch input into logits or token probabilities.
- Loss encodes the optimization objective (cross-entropy, MSE, contrastive, etc.).
- Metrics are not always losses: optimize one thing, report several.

A common anti-pattern is optimizing a proxy loss that does not match business KPI.



Backward Pass and Optimization

- Autograd computes gradients automatically from loss to parameters.
- Optimizers (AdamW, SGD) update weights using gradients.
- Schedulers adjust learning rate over time (warmup, cosine decay, step decay).
- Mixed precision speeds training and reduces memory footprint.



Validation and Error Analysis

Validation should answer: *is the model good enough for deployment?*

- Track both aggregate metrics and slices (rare classes, edge domains).
- Compare calibration, confidence, and failure modes.
- Keep a fixed benchmark subset for stable trend monitoring.

Training loss without structured error analysis is misleading.



Experiment Tracking and Reproducibility

- Log config, git revision, dataset version, metrics, artifacts.
- Save checkpoints and best model according to validation target.
- Use systems such as ClearML/MLflow/W&B to compare runs.

Reproducibility means you can recreate the same run months later.

Hyperparameters That Matter Most

Group	Examples and effect
Optimization	Learning rate, weight decay, optimizer type; strongest impact on convergence
Batching	Batch size, accumulation steps; trade-off between stability and speed
Regularization	Dropout, label smoothing, augmentation; controls overfitting
Schedule	Warmup ratio, max epochs, early stopping; affects final generalization



Why Ray for Training Workloads

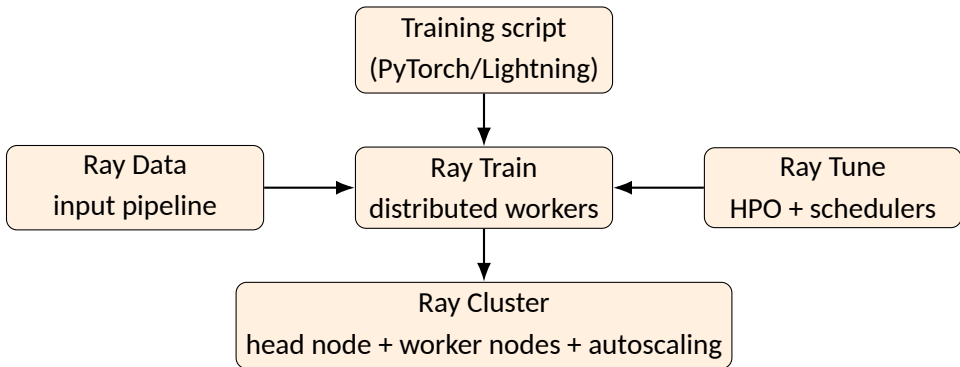
Ray is a distributed execution framework for Python workloads.

- Scale one script from laptop to cluster with minimal code changes.
- Ray Train: distributed model training (data parallel and fault-tolerant loops).
- Ray Tune: large-scale hyperparameter search and scheduler-based pruning.
- Ray Data: scalable data ingestion and preprocessing pipeline.

Ray is useful when experiments outgrow a single machine.



Ray in the Training Stack





Ray Concepts to Remember

- **Tasks:** parallel stateless functions.
- **Actors:** stateful workers for long-lived services.
- **Object Store:** shared-memory transport for large tensors and batches.
- **Schedulers:** resource-aware placement (CPU/GPU/custom resources).

These abstractions let you scale preprocessing, training, and HPO consistently.

Ray Tune for Hyperparameter Search

- Define search space (learning rate, batch size, dropout, optimizer).
- Run many trials in parallel across cluster resources.
- Use pruning schedulers (e.g., ASHA) to stop weak trials early.
- Keep top checkpoints and export best configuration.

Result: faster iteration and better utilization of available GPUs.



PyTorch Distributed: What and When

- **torch.distributed (DDP)** is the default baseline for multi-node training in both industry and academia.
- Most common CPU backend: gloo; for GPUs in production: nccl.
- Typical launch pattern: torchrun + one process per device (or one per CPU node in demos).
- Best fit: stable training jobs where you control infrastructure and need maximum performance.

Rule of thumb: start from DDP mental model first, then introduce Ray for orchestration and scale management.



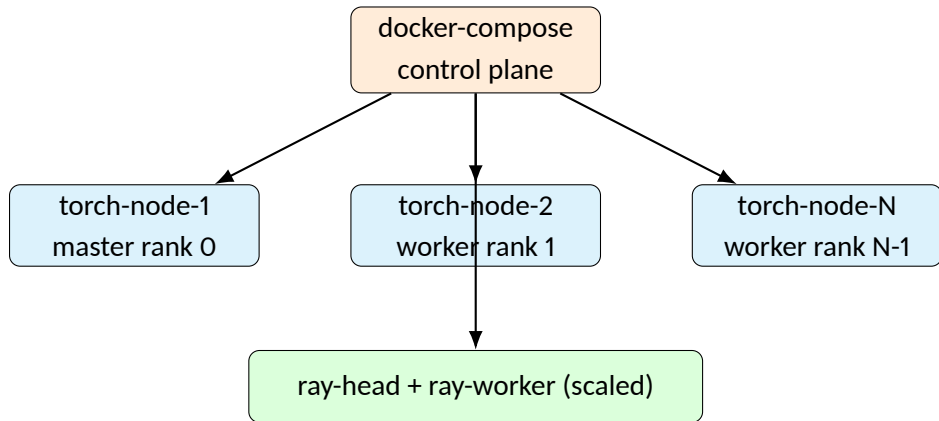
DDP vs Ray Train: Practical Choice

Aspect	PyTorch Distributed	Ray Train
Abstraction	Low-level, explicit control	Higher-level orchestration
Setup	Manual launch and env setup	Unified cluster/runtime layer
Fault handling	Mostly manual	Built-in recovery patterns
Best for	Peak performance, custom loops	Many experiments, elastic scaling

teams combine both: DDP-style training logic wrapped by Ray execution.

In practice

CPU Cluster Emulation with Docker Compose



- `WORLD_SIZE=3 docker compose ... --scale torch-node=3`

- `docker compose ... --scale ray-worker=3`



When to Show Demo in This Lecture

Recommended pacing for a 90-minute class:

1. **After pipeline basics (minute 20–25):** run quick YOLO CPU fine-tune (1 epoch).
2. **After reproducibility section (minute 40–45):** show ClearML run page and tracked artifacts.
3. **After distributed theory (minute 60–70):** run Docker-based DDP and then Ray worker scaling.

Keep each live demo under 3–5 minutes and prepare screenshots as fallback.



Mock Slide: What to Insert (System Architecture)

Mock image placeholder

Target image: **"End-to-end ML training architecture"**

Show data source, preprocessing, feature store, training workers, validation service, model registry, and deployment endpoint. Add arrows for offline training flow and online inference flow in different colors.



Mock Slide: What to Insert (Ray Cluster)

Mock image placeholder

Target image: **"Ray cluster topology for training"**

Draw one head node and 3–6 worker nodes with CPU/GPU labels. Visualize task queue, object store, and autoscaler. Add a small legend: blue for data, orange for training, green for tuning trials.



Mock Slide: What to Insert (Live Demo Setup)

Mock image placeholder

Target image: **"Classroom live-demo command map"**

Create a terminal-style infographic with 3 blocks: (1) YOLO fine-tune command, (2) ClearML dashboard screenshot area, (3) Docker scale commands for torch-node and ray-worker. Add expected runtime labels: 2-5 min each.



Tools Ecosystem

- **Modeling:** PyTorch, Lightning, Hugging Face Transformers.
- **Tracking:** ClearML (primary) or MLflow for metrics, configs, and artifacts.
- **Scaling:** Ray for distributed data processing, training, and HPO.
- **Serving:** model registry + deployment pipeline with rollback strategy.



Conclusion

Training is engineering orchestration, not only gradient updates.

- Understand each pipeline component and its failure modes.
- Optimize data quality and reproducibility before model complexity.
- Use Ray when scale and experiment throughput become bottlenecks.

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521